esprit
Se former autrement

LINQ

Cours présenté par Tarek Ayari

LINQ

PLAN

- Introduction
- Exemple
- Linq to Objects
- Méthodes de selection
- Méthodes d'agrégation

❑INTRODUCTION

LINQ?

- **Language-INtegrated Query**
- C'est un composant du .NET Standard
- Il est inspire par le langage SQL
- Il ajoute de grandes capacités d'interrogation sur des données aux langages .NET
- s'applique sur les listes, les objets, les fichiers XML, les entités, les bases de données relationnelles…

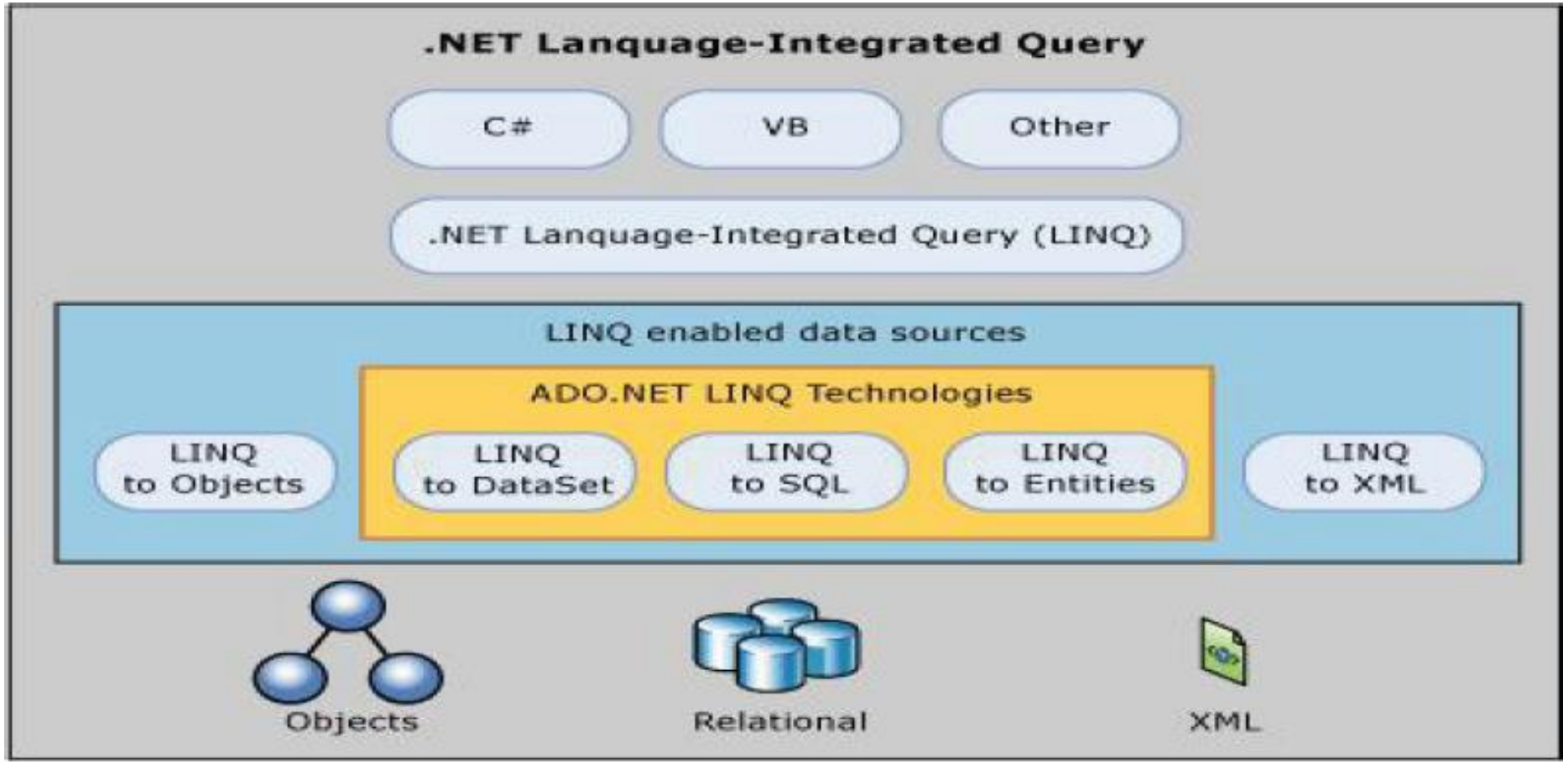
❑INTRODUCTION

LINQ?

- Le langage LINQ offre un modèle de requetage qui fait abstraction des sources de données en manipulant des objets.
- Les requêtes LINQ suivent le même modèle pour interroger que ce soit pour interroger des bases de données SQL, des documents XML, des collections .Net ou n'importe quel format pour lequel le provider LINQ est disponible

❑INTRODUCTION

Quelles sources de données sont autorisées?



ADO.NET: ActiveX Data Object

❑INTRODUCTION

Deux formes d’ écriture pour LINQ?

- La première : sous forme de requête
- La deuxième : sous forme de lambda expression

❑INTRODUCTION

- Dans le langage LINQ, l'exécution de la requête est indépendante de la requête en soit.
- En d'autres termes, écrire une requête ne déclenche pas la récupération des données, pour ce faire il faut exécuter la requête.
- La figure suivante illustre l'opération de requêtage:

❑INTRODUCTION

Etapes

Toutes les opérations de requêtage LINQ se déroulent en trois étapes :

1. Spécifier une source de données
2. Définir l'expression de la requête (Query Expression)
3. Exécuter la requête

❑EXEMPLE

Exemple

```
// data source
int[] numbers = new int[8]{1, 2, 3, 4, 5, 6, 7, 8};

// query expression
var numQuery =
    from num in numbers
    where (num % 2) == 0
    select num;

// query execution in foreach
foreach (int num in numQuery)
{
    Console.Write("{0} ", num);
}
```

Affiche 2 4 6 8

❑EXEMPLE

Explication

- `from num in numbers` : `numQuery` contiendra toute valeur `num` appartenant au tableau `numbers`
- `where (num % 2) == 0` : définit la condition de sélection
- `select num` : précise ce que l'on veut sélectionner (ici on n'a que `num`

Remarque

- La requête est exécutée seulement dans le `foreach`
- L'ordre des clauses `from, where, select` est important
- `numQuery` est de type `IEnumerable<int>`, on peut donc faire :

❑EXEMPLE

Exemple avec expression Lambda

```
int[] numbers = new int[8]{1, 2, 3, 4, 5, 6, 7, 8};

IEnumerable<int> numQuery = numbers
      .Where(elt => elt % 2 == 0);

foreach (int num in numQuery)
{
    Console.Write("{0} ", num);
}
```

❑EXEMPLE

Il est aussi possible de définir plusieurs conditions

```
int[] numbers = new int[8]{1, 2, 3, 4, 5, 6, 7, 8};
var numQuery =
    from num in numbers
    where (num % 2) == 0 && (num > 5 || num < 3)
    select num;

foreach (int num in numQuery)
{
    Console.Write("{0} ", num);
}
// affiche 2 6 8
```

❑EXEMPLE

Avec les expressions Lambdas

```
int[] numbers = new int[8]{1, 2, 3, 4, 5, 6, 7, 8};
IEnumerable<int> numQuery = numbers
      .Where(elt => elt % 2 == 0 && (elt > 5 || elt < 3));

foreach (int num in numQuery)
{
    Console.Write("{0} ", num);
}
// affiche 2 6 8
```

❑LINQ TO OBJECT

Quoi comme objet?

- Il s'agit de tout objet C#
  - tableau
  - collection
  - Liste
  - …

❑LINQ TO OBJECT

Étant donnée la classe `Personne` suivante

```
class Personne
{
    public Personne(int num, string nom, string prenom,
       int budget)
    {
        this.Nom = nom;
        this.Prenom = prenom;
        this.Num = num;
        this.Budget = budget;
    }
    public int Num { get; set; }
    public string Nom { get; set; }
    public string Prenom { get; set; }
    public int Budget { get; set; }
}
```

❑LINQ TO OBJECT

Créons une liste à partir de cette classe `Personne`

```
List<Personne> personnes = new List<Personne>()
{
    new Personne(1,"wick","john",500),
    new Personne(2,"abruzzi","john",700),
    new Personne(3,"dalton","jack",300),
    new Personne(4,"white","mike",800),
};
```

❑LINQ TO OBJECT

Pour afficher les personnes qui ont un budget supérieur à 500

```
IEnumerable <Personne> persoQuery =
    from perso in personnes
    where ( perso.Budget > 500 )
    select perso;
foreach ( Personne perso in persoQuery )
{
    Console.WriteLine($"Je m'appelle { perso.Prenom } {
       perso.Nom }");
}
```

❑LINQ TO OBJECT

Autre solution avec l'expression Lambda

```
var persoQuery = personnes.Where(elt => elt.Budget >
    500);

foreach ( Personne perso in persoQuery )
{
    Console.WriteLine($"Je m'appelle { perso.Prenom
       } { perso.Nom }");
}
```

❑LINQ TO OBJECT

Si on veut sélectionner seulement quelques attributs (et les renommer)

```
var persoQuery =
    from perso in personnes
    where (perso.Budget > 500)
    select new {
        Name = perso.Nom,
        Number = perso.Num
    };

foreach ( var perso in persoQuery )
{
    Console.WriteLine($"Je m'appelle { perso.Name } {
       perso.Number }");
}
```

❑LINQ TO OBJECT

Autre solution avec l'expression Lambda

```
var persoQuery = personnes
    .Where(elt => elt.Budget > 500)
    .Select(elt => new {
        Name = elt.Nom,
        Number = elt.Num
    }
);

foreach ( var perso in persoQuery )
{
    Console.WriteLine($"Je m'appelle { perso.Name } {
      perso.Number }");
}
```

❑LINQ TO OBJECT

Autres clauses de requête LINQ

- `orderby` : permet de trier dans l'ordre croissant ou décroissant.
- `group ... by ... into` : permet de retourner une séquence composée de clé (correspondant au critère du groupement) et valeur (correspondant à l'objet)
- `join ... on ... equals` : permet de faire des jointures entre deux énumérables
- ...

❑LINQ TO OBJECT

Exemple avec `orderby`

```
IEnumerable<Personne> persoQuery =
     from perso in personnes
     where ( perso.Budget >= 500 )
     orderby perso.Nom ascending // ou descending
     select perso;
foreach ( Personne perso in persoQuery )
{
  Console.WriteLine($"Je m'appelle { perso.Prenom }
     { perso.Nom }");
}
```

❑LINQ TO OBJECT

La solution avec l'expression Lambda

```
IEnumerable<Personne> persoQuery =
    personnes.Where(perso => perso.Budget >= 500)
    .OrderBy(perso => perso.Nom);

foreach ( Personne perso in persoQuery )
{
  Console.WriteLine($"Je m'appelle { perso.Prenom }
     { perso.Nom }");
}
```

❑LINQ TO OBJECT

- **Group BY**

```
//Expression Lambda
var GroupByPrenom = personnes.Where(perso=>perso.budget>=500)
.GroupBy(perso => perso.prenom);

//Forme requete
IEnumerable<IGrouping<string, Personne>> GroupByPR = (from perso in
personnes Where (perso=>perso.budget>=500)
group perso by perso.prenom);
```

❑LINQ TO OBJECT

- Group BY

```
foreach(var group in GroupByPrenom )
        {
            Console.WriteLine(group.Key +" : " + group.Count());

            //Iterate through each Person of a group
            foreach(var person in group)
            {
                Console.WriteLine("  Nom :" + person.Nom + ", Prenom: " + student.Prenom);
            }
        }
```

❑FILTERING OPERATOR - OFTYPE

Example: OfType operator in C#

```
IList mixedList = new ArrayList();
mixedList.Add(0);
mixedList.Add("One");
mixedList.Add("Two");
mixedList.Add(3);


var stringResult = from s in mixedList.OfType<string>()
                   select s;

var intResult = from s in mixedList.OfType<int>()
                select s;
```

```
One
Two
0
3
```

❑MÉTHODES DE SÉLECTION

- **First():** Renvoie le premier élément d'une collection ou le premier élément qui satisfait une condition.

- **FirstOrDefault():** Renvoie le premier élément d'une collection ou le premier élément qui satisfait une condition. Renvoie une valeur par défaut si l'index est hors plage.

- **Single():** Renvoie le seul élément d'une collection ou le seul élément qui satisfait à une condition.

- **SingleOrDefault():** Identique à Single, sauf qu'il renvoie une valeur par défaut d'un type générique spécifié, au lieu de lever une exception

❑MÉTHODES DE SÉLECTION

- **Last():** Renvoie le dernier élément d'une collection ou le dernier élément qui satisfait à une condition.

- **LastOrDefault():** Renvoie le dernier élément d'une collection ou le dernier élément qui satisfait à une condition. Renvoie une valeur par défaut si aucun élément de ce type n'existe.

- **Skip():** Pour indiquer le nombre d' élément `a ignorer de la sélection.

- **Take():** Pour indiquer le nombre d' élément `a garder de la sélection.

❑MÉTHODES D'AGRÉGATION

- **Average():** calcule la moyenne des éléments numériques de la collection.
- **Sum() :** calcule la somme des éléments numériques de la collection.
- **Max():** renvoie le plus grand élément numérique d'une collection.
- **Count:** renvoie le nombre d'éléments de la collection ou le nombre d'éléments qui ont satisfait à la condition donnée.